

UNIT – II Leetcode Exercise

1. Find Users with Valid E-Mails

Problem Summary

You are given a table named Users.

Column Name	Type
user_id	int
Name	varchar
mail	varchar(50)

- user_id is the primary key.
- Each row contains the ID and name (which is an e-mail address) of a user.

You need to find users whose e-mail addresses are valid.

Valid E-mail Rules

A valid e-mail:

1. Starts with a letter (a-z or A-Z)
2. Followed by:
 - letters (a-z, A-Z)
 - digits (0-9)
 - underscore (_)
 - period (.)
 - hyphen (-)

3. Ends with:

@leetcode.com

Task:

Write an SQL query to find all users having valid e-mail addresses.

Return the result table in any order.

</> Code

MySQL  Auto

```
2 SELECT
3   user_id, name, mail
4 FROM users
5 WHERE
6   REGEXP_like(mail, '^[a-zA-Z][a-zA-Z0-9_-.]*@leetcode\\.com$', 'c');
7
```

Saved

☒ Testcase |  Test Result

☒ Case 1

Input

Users =

user_id	name	mail
1	Winston	winston@leetcode.com
2	Jonathan	jonathanisgreat
3	Annabelle	bella@leetcode.com
4	Sally	sally.come@leetcode.com
5	Marwan	quarz#2020@leetcode.com
6	David	david69@gmail.com
7	Shapiro	.shapo@leetcode.com

 View less

Output

user_id	name	mail
1	Winston	winston@leetcode.com
3	Annabelle	bella@leetcode.com
4	Sally	sally.come@leetcode.com

Explanation:

- REGEXP – Used for pattern matching in MySQL.
- ^ and \$ – Used as start and end anchors.
- Character Classes [] – Used to restrict the allowed characters.
- Quantifier – Represents zero or more repetitions.
- Escaping Special Characters (.) – Used to match a literal dot (.).

2. Patients With a Condition

Problem Summary

You are given a table named Patients.

• patient_id int
• patient_name varchar
• conditions varchar
• patient_id is the primary key.

Important Rule

A patient is considered valid if:

- DIAB1 appears at the beginning of the string, or
- DIAB1 appears after a space.

Task

Write an SQL query to report the patient ID, patient name, and conditions of patients who

have Type I Diabetes (DIAB1).

Return the result table in any order.

</> Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 select * from Patients
3 where conditions like 'DIAB1%' or conditions like '% DIAB1%'
4
```

Saved

☒ Testcase |  Test Result

Accepted Runtime: 82 ms

☒ Case 1

Input

Patients =

patient_id	patient_name	conditions
1	Daniel	YFEV COUGH
2	Alice	
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100
5	Alain	DIAB201

Output

patient_id	patient_name	conditions
3	Bob	DIAB100 MYOP
4	George	ACNE DIAB100

Explanation:

Bob and George both have a condition that starts with DIAB1.

3. Second Highest Salary

Problem Summary

You are given an Employee table.

• id int
• salary int
• id is the primary key.

- Each row contains the salary of an employee.
- Salaries may contain duplicates.
- Your task is to find the second highest distinct salary.
- If there is no second highest salary, return NULL.

Task

Write an SQL query to report the second highest distinct salary from the Employee table.

`</>` Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT (
3     SELECT DISTINCT salary
4     FROM Employee
5     ORDER BY salary DESC
6     LIMIT 1 OFFSET 1
7 ) AS SecondHighestSalary;
8
```

Saved

☒ Testcase |  Test Result

Accepted Runtime: 96 ms

☒ Case 1

☒ Case 2

Input

Employee =

id	salary
1	100
2	200
3	300

Output

SecondHighestSalary
200

Explanation:

We wrap the query in an outer `SELECT (...) AS SecondHighestSalary`. This is a crucial trick. If the inner query finds no rows (because no 2nd salary exists), it returns an empty set. The outer `SELECT` catches this and correctly outputs a single row with `NULL`, as required.

4.Customer Placing the Largest Number of Orders

Problem Summary

You are given an Orders table.

• order_number int
• customer_number int

- order_number is unique.
- Each row represents one order placed by a customer.
- A customer can place multiple orders.

Your task is to find the customer who placed the largest number of orders.

The problem guarantees that:

There is exactly one customer who has placed the maximum number of orders.

Task

Write an SQL query to find the customer_number of the customer who has placed the largest number of orders.

Return the result table in any order.

 Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT customer_number
3 FROM Orders
4 GROUP BY customer_number
5 ORDER BY COUNT(*) DESC
6 LIMIT 1;
7
```

Saved

☒ Testcase |  Test Result

Accepted Runtime: 66 ms

☒ Case 1

Input

orders =

order_number	customer_number
1	1
2	2
3	3
4	3

Output

customer_number
3

Explanation:

The customer with number 3 has two orders, which is greater than either customer 1 or 2 because each of them only has one order. So the result is customer_number 3.

5. Product Sales Analysis III

Problem Summary

You are given two tables, but only the Sales table is needed for this problem.

Sales Table

• sale_id int
• product_id int
• year int
• quantity int
• price int

- (sale_id, year) is the primary key.
- Each row represents a sale of a product in a particular year.

Objective

For each product, find:

1. The first year in which the product was sold.
2. The quantity sold in that year.
3. The price of the product in that year.

Task

Write an SQL query to report:

- product_id
- first_year
- quantity
- price

for the first year in which each product was sold.

Return the result table in any order.

</> Code

MySQL   Auto

```
1 # Write your MySQL query statement below
2 SELECT product_id, year first_year, quantity, price FROM sales
3 WHERE
4     (product_id, year) IN (
5         SELECT
6             product_id,
7             MIN(year)
8         FROM sales
9         GROUP BY product_id
10    );
```

Saved

☒ Testcase |  Test Result

Accepted Runtime: 78 ms

☒ Case 1

Input

Sales =

sale_id	product_id	year	quantity	price
1	100	2008	10	5000
2	100	2009	12	5000
7	200	2011	15	9000

Output

product_id	first_year	quantity	price
100	2008	10	5000
200	2011	15	9000

Explanation:

This query finds the **first year** each product was sold. It then displays the **product ID, year, quantity, and price** for that first year.